QueryMind AI

Natural Language Database Assistant

Project Proposal Document – MSc IT Application Development Project

Team Member	Student ID	Responsibility Area
Deep Soni	202512089	Deployment and Post Integration Testing
Ishpreet Kaur	202512104	Backend / Database and Query Execution
Neel Shah	202512114	Backend / Database and Query Execution
Harsh Rathod	202512086	Security and Query Validation
Nishit Gal	202512002	Security and Query Validation
Parshwa Jain	202512096	Backend / Database and Query Execution
Heer Sachdev	202512117	Frontend/UI and UX
Bansari Akhani	202512032	Frontend/UI and UX

1. Project Title and Short Description

QueryMind AI is a web based assistant that lets a person retrieve information from a relational database by typing a question in plain English or Hinglish instead of writing SQL. The system interprets the question, generates the corresponding SQL statement against the connected database schema, executes it under a set of safety checks, and returns the result as a table and a brief summary about result in text . The idea of translating natural language into SQL is not new and several commercial and open source tools already attempt this, so the contribution of this project lies in building a complete, working, end to end system: schema aware query generation, support for Hinglish phrasing that is common in Indian workplaces, a validation layer that prevents unsafe queries from ever reaching the database, and a measurable evaluation of how well the generated SQL actually matches what the user asked for.

2. Application Type

QueryMind AI will be built as a web application. The primary interface is a browser based chat window used by end users to ask questions and view results, and a separate browser based admin panel used by a database administrator to connect databases, manage schema mappings, and review usage. No mobile client is planned for this project since the target use cases (office and back office data access) are desktop first, and a browser based interface keeps the scope realistic.

3. Motivation and Problem Statement

In most organisations, the people who actually need answers from a database, such as an HR executive checking attendance records, a hospital administrator counting patients by department, or an inventory manager checking stock levels, do not know SQL. They either depend on a developer or analyst to write a query for them or fall back on static, pre built reports that may not answer the exact question they have. This creates delay and adds load on technical staff for what are often simple lookups. Text to SQL systems address part of this problem, but many available tools assume clean English input, do not account for the way people in Indian workplaces mix English and Hindi while typing (Hinglish), and give limited attention to what happens if the generated SQL is wrong, unsafe, or run against the wrong table. QueryMind AI is our attempt at building a reasonably complete version of this idea, with explicit attention to safety, multiple database support, and evaluation of correctness rather than just a working demo.

4. Target Users and Use Cases

The system is aimed at people who work with data regularly but are not expected to write SQL:

1. HR or payroll staff checking attendance, leave balance, or salary records for a department
2. Hospital or clinic administrative staff retrieving patient counts, appointment schedules, or ward occupancy
3. Inventory or warehouse staff checking stock levels, reorder points, and supplier wise stock reports
4. Academic institution staff retrieving admission, attendance, or examination statistics
5. Small business owners checking sales figures and exporting them for accounting purposes
6. A database administrator who configures which databases are connected, which tables are exposed, and which roles can access what, through the admin panel

5. Scope and High Level Features

5.1 Core Modules

1. **Voice & Text Query Input** – Accept natural language queries through text or voice in English and Hinglish.

2. **Natural Language Processing (NLP)** – Understand user intent and identify the required database operation.

3. **Automatic Schema Discovery** – Dynamically read database schema, tables, columns, and relationships.

4. **LLM-Based Text-to-SQL Generation** – Convert natural language queries into optimized SQL using a LLM.

5. **SQL Validation & Security** – Validate generated SQL, block destructive operations, and prevent SQL injection.

6. **Multi-Database Connectivity** – Support seamless connection with MySQL , MongoDb and PostgreSQL databases.

7. **Query Review & Execution** – Display generated SQL for user approval before secure execution.

8. **Conversational Follow-up Support** – Allow context-aware follow-up queries within the same session.

9. **Authentication & Role-Based Access** – Manage secure login with Admin, Analyst, and Viewer roles.

10. **Admin Dashboard** – Manage database connections, schema visibility, users, and query logs.

11. **Query History** – Store previously executed natural language queries and generated SQL for each user.

12. **Testing & Evaluation** – Evaluate system accuracy using English/Hinglish query datasets and security test cases.

5.2 Future Scope (not part of this semester's deliverable)

- Support additional regional and international languages.
- Add support for more databases such as MongoDB, Oracle, SQL Server, and SQLite.
- Implement persistent AI chat memory across multiple sessions.
- Integrate with BI tools such as Power BI and Tableau.
- Provide AI-generated database insights and recommendations.
- Develop a real-time conversational voice assistant with speech responses.
- Add automated report scheduling and email sharing of query results.

6. System Overview

The system is organised into five layers. The web client layer is the chat interface and admin panel used by end users and administrators. The backend and API layer, built on FastAPI, receives requests, coordinates the other layers, and returns responses. The AI and NLP processing layer takes the user's natural language input, resolves it against the schema, and produces a candidate SQL statement, which is then passed through the validation layer before reaching the database connectivity layer, which abstracts over MySQL and PostgreSQL and actually executes the approved query. Results flow back up through the API layer to the client, where they are rendered as tables, charts, or exported files. This layering is what allows the eight work areas described in section 8 to be developed largely in parallel, with the API layer acting as the contract between them.

7. Technology Stack

Layer	Technology
Frontend	HTML5, Tailwind CSS , CSS3, ReactJS
Backend and API	Python Django Framework, FastAPI + REST API
AI and NLP / Text to SQL	LangChain, with an LLM accessed through API (GPT, Gemini, or Llama, subject to access and cost)
Database engines supported	MongoDB
Database access layer	SQLAlchemy(Python library)
Authentication	JWT based authentication with role based access control
Testing	Pytest for backend, manual and scripted UI testing
Deployment	Hosting on a low cost platform such as AWS
Version control	Git and GitHub, with a GitHub Actions workflow for checks

8. Roles and Responsibilities (Eight Team Members)

The benchmark shared for this project is 200 person hours per member over the semester, roughly 1600 person hours for the eight member team. Rather than a group of general purpose developers, the modules described in section 5 are grouped into eight distinct responsibility areas so that each member owns a substantial, independently gradable piece of the system. Detailed task level breakdowns and hour estimates against this benchmark will be developed in the Project Plan submission; this section establishes the division of ownership.

Member	Responsibility Area	Core Tasks
Deep Soni	Deployment and Post Integration Testing	AWS/cloud deployment, production environment setup, CI/CD configuration, post-integration testing, deployment verification, system monitoring, performance checks, and resolving deployment-related issues
Ishpreet Kaur	Backend / Database and Query Execution	Database schema discovery and integration, metadata handling, MySQL and PostgreSQL connectivity, database abstraction layer, query execution support, and database-related backend development
Neel Shah	Backend / Database and Query Execution	FastAPI backend development, API endpoints, request and response pipeline, query execution engine, timeout and row-limit handling, and integration of backend services with the database layer
Harsh Rathod	Security and Query Validation	SQL query sanitisation, statement whitelisting, injection pattern detection, query validation, security rules, access-related query checks, and development of security test cases
Nishit Gal	Security and Query Validation	Query security validation, malicious-input detection, SQL injection testing, validation of generated SQL queries, security test-case implementation, and assisting with secure query-processing workflows
Parshwa Jain	Backend / Database and Query Execution	Backend query-processing components, database interaction, query execution and result handling, API-to-database integration, error handling, and optimisation of database query workflows
Heer Sachdev	Frontend/UI and UX	Chat interface development, query input experience, frontend navigation, result rendering, user interaction design, responsive UI components, and integration of frontend with backend APIs
Bansari Akhani	Frontend/UI and UX	Frontend component development, chat and query interface, result presentation, user experience improvements, frontend-backend integration, responsive design, and usability testing

9. Timeline and Milestones

Period	Milestone
27 Jul – 5 Aug 2026	Problem identification, project scoping, requirement analysis, and finalisation of the project proposal
6 Aug – 9 Aug 2026	Project module breakdown, allocation of responsibilities among team members, and project board setup
10 Aug – 16 Aug 2026	Detailed functional and technical requirements, definition of user stories for the six query categories, and preparation of initial UI wireframes
17 Aug – 27 Aug 2026	Finalisation of system architecture, database architecture and schema requirements, FastAPI REST API design, and definition of API contracts
28 Aug – 11 Oct 2026	**Core Development Phase:** development of database integration, schema discovery, Text-to-SQL pipeline, FastAPI backend, SQL validation and security layer, and initial frontend interface, with weekly progress reviews
28 Aug – 19 Oct 2026	**Mid-Semester Demonstration:** demonstration of a partially integrated system covering the complete flow from natural-language query input to SQL generation, validation, execution, and result display
20 Oct – 26 Oct 2026	Development and integration of frontend features, result visualisation, authentication and role-based access, follow-up query handling, and refinement of backend and security components
27 Oct – 01 Nov 2026	**Testing Phase:** execution of unit and integration tests, preparation of realistic test databases, security and SQL validation testing, and verification of frontend-backend integration
2 Nov – 09 Nov 2026	Bug fixing, performance optimisation, deployment preparation, evaluation using the defined test dataset, and recording of system accuracy and performance results
10 Nov – 13 Nov 2026	**Final Integration and Deployment:** feature freeze, final system integration, post-integration testing, AWS deployment, and production verification
14 Nov – 15 Nov 2026	Final project report submission
16 Nov – 18 Nov 2026	Final presentation, live system demonstration, project poster preparation, and submission of final project deliverables

10. Expected Outcomes

By the end of the semester, we expect to develop and deploy a web-based AI-powered database assistant that enables users to interact with relational databases using natural language through both text and voice input. The system will support English and Hinglish queries, automatically discover the connected database schema, generate SQL queries using a Large Language Model (LLM), validate them for security, and allow users to review and approve the generated SQL before execution. The application will support MySQL and PostgreSQL databases, present query results in tabular and graphical formats, and provide options to export results in Pdf formats.

Additionally, the project is expected to provide secure user authentication with role-based access control, conversational follow-up support, query history management, and an administrative dashboard for database management and monitoring. The final deliverables will include the complete source code, deployed application, system documentation, API documentation, user manual, testing and evaluation reports, and performance analysis demonstrating the system's accuracy, reliability, usability, and security in converting natural language queries into executable SQL statements.